

<<

4.5 页面UI与适配需求

5. 数据与接口规范（Mock API + 本地...

5.1 整体接口设计规范

5.2 全部Mock API接口清单

5.2.1 计时器配置接口

5.2.2 学习任务接口

5.2.3 学习打卡接口

5.2.4 数据统计接口

5.3 接口对应数据结构

5.1 任务列表数据结构

5.2 打卡记录数据结构

5.3 计时器配置数据结构

5.4 双层数据联动逻辑

6. 非功能性需求

6.1 性能需求

6.2 稳定性需求

6.3 可拓展性需求

6.4 可维护性需求

7. 开发技术栈约束

8. 项目交付标准

• 计时运行流畅，无卡顿、时间漂移问题；

• 接口请求高效稳定，异步数据渲染无卡顿、无数据错乱问题。

6.2 稳定性需求

• 完善的异常容错机制，杜绝非法输入导致功能报错；

• 定时器无叠加、无内存泄漏，长期运行稳定；

• 接口请求完善异常处理（超时、失败、重复请求拦截），本地存储兜底读写安全，无数据丢失、错乱问题。

6.3 可拓展性需求

• 代码模块化、组件化拆分清晰，支持后续拓展云端存储、用户登录、数据导出、学习周报生成等功能；

• 数据结构设计通用，支持后续新增统计维度、任务分类等需求。

6.4 可维护性需求

• 代码命名规范、注释完整，层级清晰；

• 功能单一职责，模块解耦，便于调试、修改、重构；

• 适配AI辅助接口开发、联调、报错排查，兼容主流AI编码工具生成请求、Mock数据、容错代码逻辑。

7. 开发技术栈约束

• 基础阶段：HTML5 + CSS3 + 原生JavaScript

• 框架阶段：Vue3（Composition API）

• 网络请求：Axios（封装通用请求工具类）

• 可视化组件：ECharts

• Apifox 用于模拟标准 RESTful 交互，LocalStorage 负责浏览器侧持久化。

• 使用AI 辅助编程

8. 项目交付标准

满足全部上述功能需求，无功能性Bug，页面适配正常，数据持久化有效，可直接在浏览器运行，最终交付：

• 完整可运行前端源码；

• 规范化、带注释的代码文件；

• 响应式适配完成的成品页面；

• 支持打包部署上线的工程化项目（可选）

↑

②

https://nankai.feishu.cn/docx/Ka1Gd9JnIoAYHxxQfQ5cVbLanbe

Page 1 of 1